Lucas Gómez · Martín Bentura
Noviembre de 2026

Valheim Server Manager en AWS

Introducción a AWS Cloud · Proyecto final · Servidores de Valheim bajo demanda

Se diseñó, desplegó y probó en AWS un sistema que administra servidores de Valheim bajo demanda. Desde un panel web se crean, se prenden y se apagan; si no queda nadie jugando se apagan solos y el mundo queda guardado en S3. Toda la infraestructura está en cuatro templates de CloudFormation. Cuesta unos 0,09 USD por hora de juego, frente a unos 65 USD por mes de una instancia prendida las 24 horas.

1. RESUMEN

- **Qué se hizo.** Un gestor de servidores de Valheim con dos partes: la que administra (S3, API Gateway, seis Lambdas, DynamoDB, EventBridge) y la que corre el juego (VPC, NLB UDP, un Auto Scaling Group de 0 a 1 por servidor, EC2 con Docker, mundos en S3).
- **Qué se probó.** Despliegue de cero sin errores, dos veces. El servidor pasa de apagado a En línea en 4 minutos; el apagado por inactividad, el apagado limpio, los backups y los avisos por Discord funcionaron sin intervención. Crear y eliminar servidores desde el panel también quedó probado.
- **Qué cuesta.** 0,09 USD por hora de servidor prendido más 0,56 USD por día de balanceador mientras exista el stack.

2. ALCANCE

Diseño, despliegue y pruebas del sistema en la cuenta de AWS Academy Learner Lab del grupo, región us-east-1.

- **Incluye:** arquitectura y decisiones, seguridad, resiliencia, escalabilidad, costos, los cuatro templates, el panel web y las pruebas.
- **No incluye:** alta disponibilidad entre regiones ni autenticación de usuarios individuales; ver la sección de limitaciones.

3. QUÉ HACE LA APLICACIÓN

FUNCIÓN	CÓMO SE RESUELVE EN AWS
Ver los servidores y su estado	Panel estático en S3 que llama a una HTTP API; la Lambda `api-handler` lee DynamoDB y los ASG.

FUNCIÓN	CÓMO SE RESUELVE EN AWS
Crear y eliminar servidores	La Lambda `provisioner` crea o borra el stack de CloudFormation del servidor (`04-servidor.yaml` en S3).
Prender y apagar	La Lambda cambia la capacidad deseada del ASG entre 0 y 1.
Dirección fija	Network Load Balancer con listeners UDP 2456 y 2457; el ASG registra la instancia.
Conservar el mundo	S3 con versionado: se restaura al arrancar, backup cada 20 minutos y al apagar.
Apagado automático	Métrica `PlayersOnline` por minuto; alarma a los 30 minutos en cero; EventBridge invoca `idle-stop`.
Apagado sin perder progreso	Lifecycle hook del ASG, Lambda `graceful-shutdown` y Run Command: detiene el juego, sube el mundo y libera el hook.
Historial, avisos y costos	Eventos en DynamoDB visibles en el panel; webhook de Discord; apagado programado a las 04:00; límite de 10 servidores creados y 8 prendidos.

4. ARQUITECTURA

El plano de control está fuera de la VPC: S3 sirve el panel, API Gateway recibe las acciones con una clave en el header, seis Lambdas ejecutan la lógica, DynamoDB guarda estado e historial, EventBridge enruta los eventos y SQS recibe lo que falla. El plano de datos es una VPC en dos zonas de disponibilidad con un NLB UDP y un ASG de mínimo 0 y máximo 1 por servidor; la instancia corre Valheim en Docker y guarda el mundo en S3.

- **Prender.** El panel llama a la API con la clave `x-api-key`; `api-handler` pone `DesiredCapacity = 1`. Si el servidor anterior todavía se está apagando responde 409.
- **Arranque.** La instancia restaura el mundo desde S3, levanta el contenedor y el ASG la registra en el NLB. `state-sync` toma el evento del ASG, marca el estado y avisa en Discord; cuando el juego responde, la instancia marca En línea.
- **Apagado.** Con 30 minutos sin jugadores la alarma dispara `idle-stop`, que pone la capacidad en 0. El lifecycle hook frena la terminación hasta que `graceful-shutdown`, por Run Command, detiene el juego y sube el mundo a S3. La misma Lambda corre por cron a las 04:00.
- **Crear y eliminar.** `provisioner` valida los datos, controla el límite y pide a CloudFormation el stack `valheim-srv-<nombre>`; `state-sync` sigue el estado del stack y deja el servidor en

Apagado al terminar. Eliminar borra el stack solo con el servidor apagado; el mundo queda en S3.

DECISIÓN	ALTERNATIVA	MOTIVO
ASG de 0 a 1 con el mundo en S3	EC2 que se detiene y arranca con su EBS	Sin costo cuando nadie juega y relanzamiento automático si falla la instancia o la AZ. A cambio, el mundo se guarda en forma explícita.
Network Load Balancer	Elastic IP	Valheim es UDP y con estado; un ALB no aplica. El NLB da DNS fijo, health checks y registro automático. Es lo más caro (16 USD por mes); una EIP cuesta 3,65.
DynamoDB, Lambdas fuera de la VPC y Gateway Endpoints	RDS, Lambdas en subred privada, NAT Gateway	Pocos ítems y acceso por clave; ninguna Lambda necesita recursos privados. Sin NAT ni Interface Endpoints, y el tráfico pesado no sale a internet.
EC2 con Docker	ECS o Fargate	Un solo contenedor con estado por servidor. Fargate cuesta cerca del doble por hora y no resuelve el estado.
CloudFormation como API para crear servidores	Crear los recursos con boto3	El mismo template sirve para la consola y para el panel; si algo falla CloudFormation deshace todo.

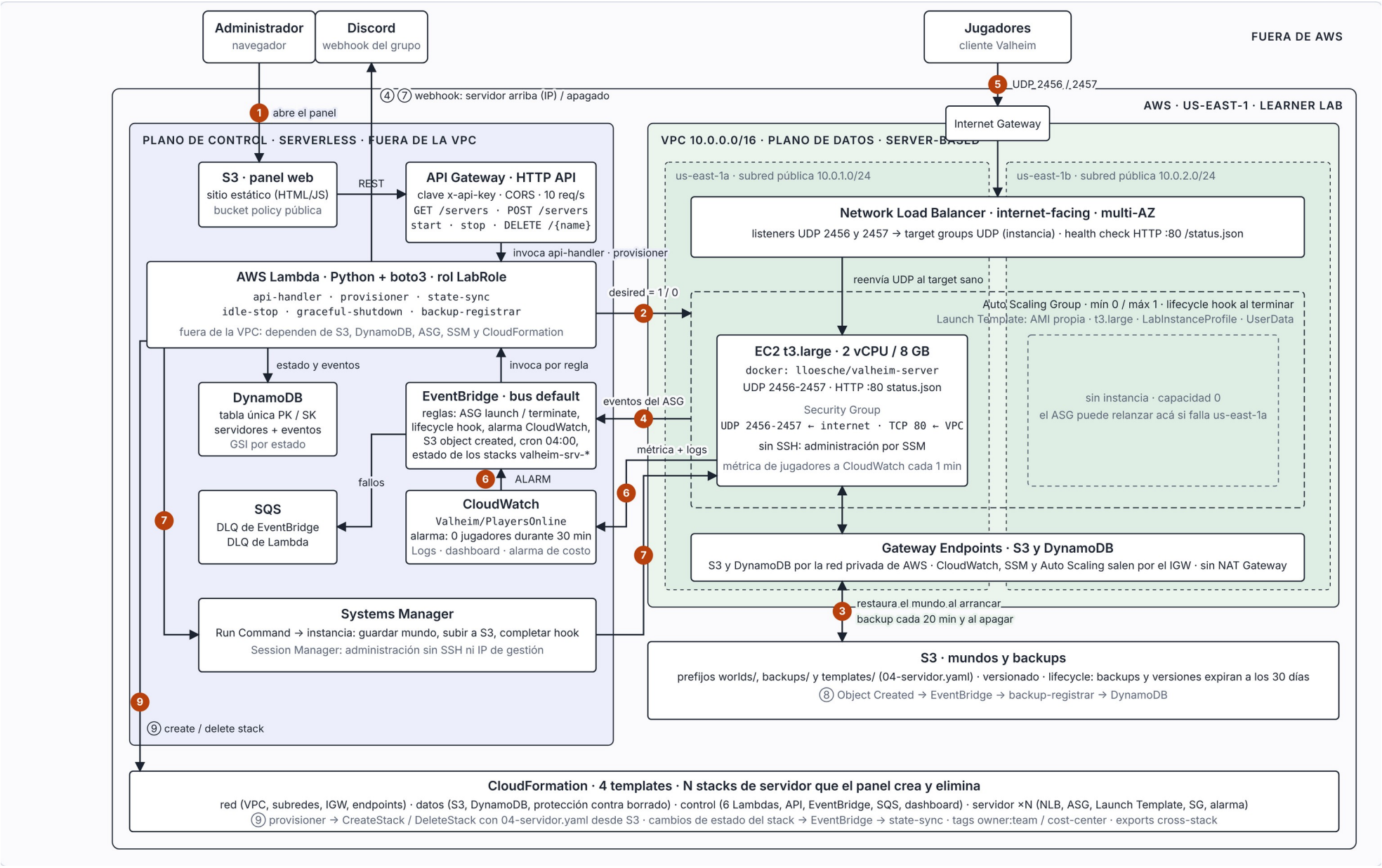


Figura 1. Componentes y flujo. A la izquierda el plano de control fuera de la VPC; a la derecha la VPC con el servidor dentro de un ASG de 0 a 1 detrás del NLB. Los números marcan el flujo principal; el 9, la creación de servidores desde el panel.

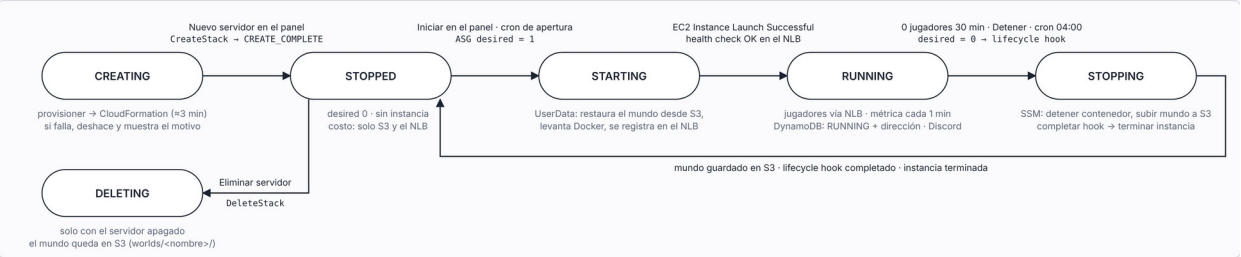


Figura 2. Ciclo de vida de un servidor. CREATING y DELETING son estados del stack de CloudFormation; los otros cuatro los escriben las Lambdas con los eventos del ASG.

5. SEGURIDAD

Lo único que entra desde internet a un recurso propio es el UDP del juego, por el NLB. El panel y la API terminan en servicios de AWS con TLS. El servidor no tiene SSH ni IP de gestión.

- **Redes públicas y privadas.** VPC 10.0.0.0/16 con una subred pública y una privada por AZ. En las públicas van solo el NLB y el servidor; las privadas no tienen salida a internet y quedan reservadas. El servidor no puede ir a una privada sin NAT Gateway (33 USD por mes) porque necesita salir a Steam.
- **Security groups.** UDP 2456-2457 desde cualquier origen y TCP 80 solo desde la VPC (health check). El resto son conexiones que inicia la instancia. La API tiene throttling de 10 solicitudes por segundo.
- **Protocolos seguros.** HTTPS en todo el plano de control. Administración por Session Manager y Run Command, sin puertos abiertos. El juego usa UDP con contraseña.
- **Encriptación e identidad.** EBS, S3 (SSE-S3, versionado, Block Public Access) y DynamoDB cifrados; TLS en todas las llamadas. Contraseña, clave de la API y webhook como parámetros `NoEcho`. Las Lambdas usan `LabRole` y la instancia `LabInstanceProfile`: el lab no permite roles de mínimo privilegio y se declara como limitación.

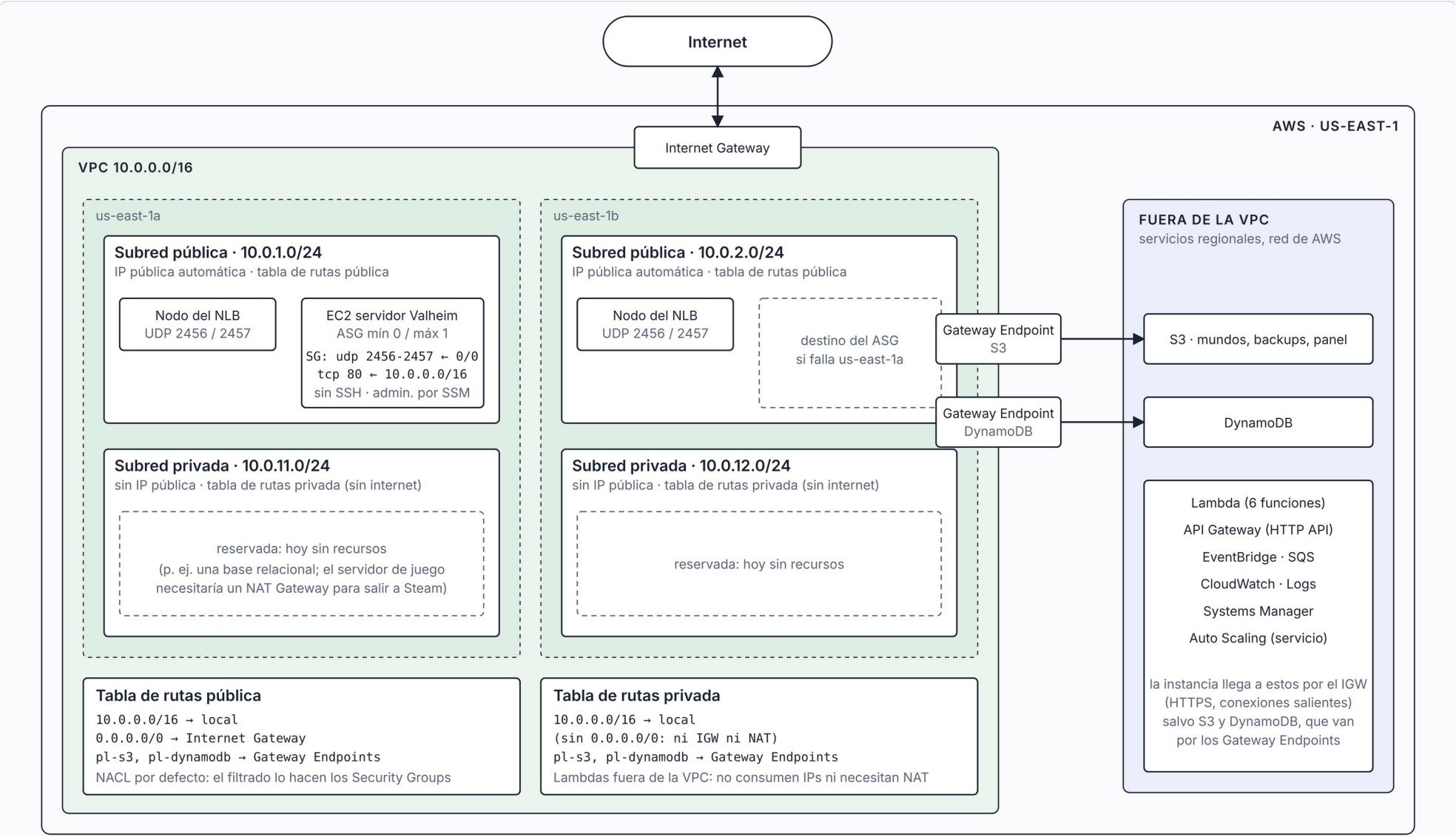


Figura 3. Redes: dos AZ con una subred pública y una privada cada una, Internet Gateway, tablas de rutas, Gateway Endpoints y los servicios regionales fuera de la VPC.

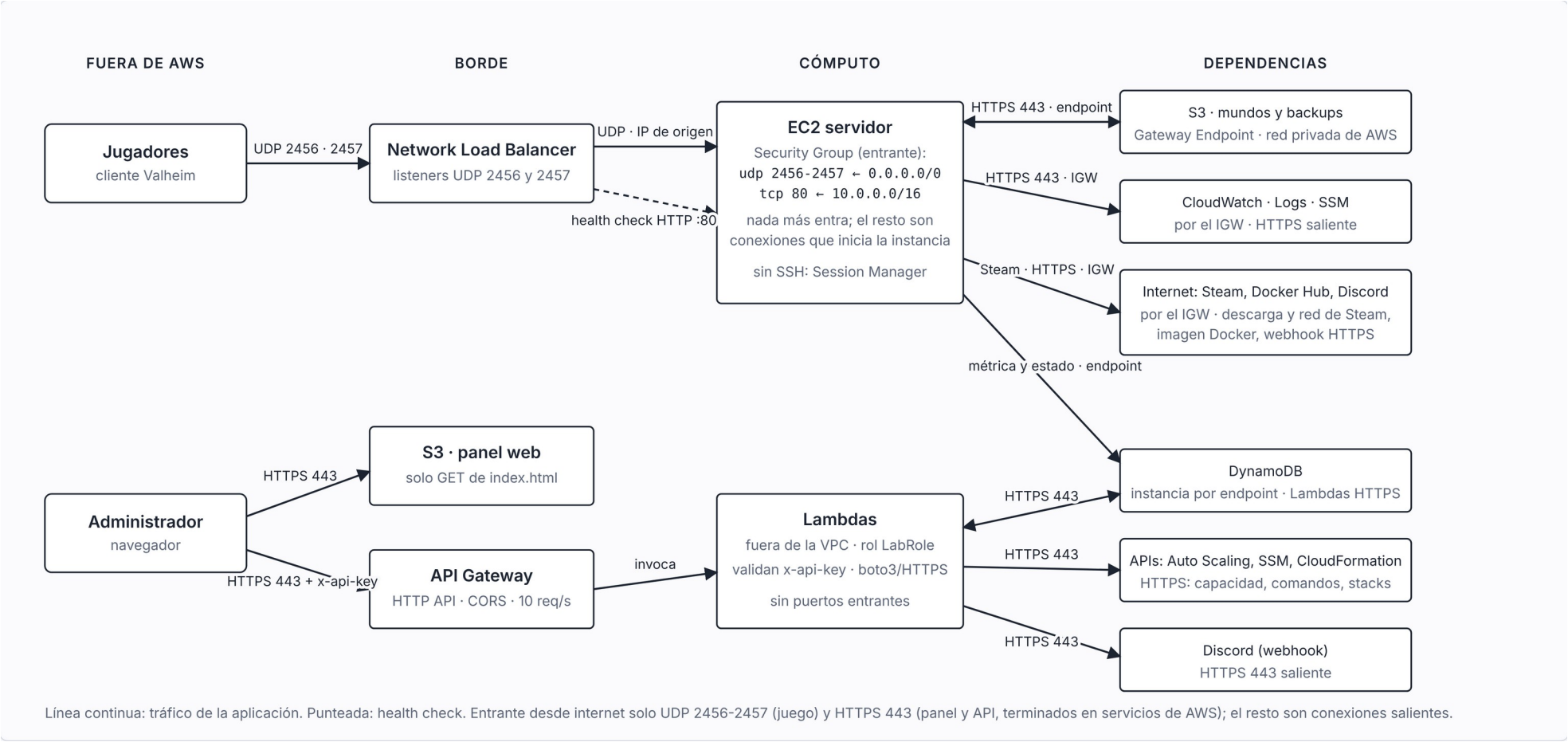


Figura 4. Tráfico: protocolos, puertos y por dónde sale cada conexión.

1 · Perímetro y red

Subredes públicas solo para lo que debe recibir tráfico: servidor y nodos del NLB.
Subredes privadas sin ruta a internet, reservadas para componentes internos.

Security Group del servidor (entrante):

```
udp 2456-2457 ← 0.0.0.0/0 (juego y consulta)
tcp 80 ← 10.0.0.0/16 (health check del NLB)
```

Sin SSH ni IP de gestión: administración por Systems Manager Session Manager.
NLB solo con listeners UDP 2456 y 2457; API con throttling de 10 req/s.
Lambdas fuera de la VPC: sin puertos entrantes, solo llamadas salientes a AWS.
NACL por defecto; el filtrado se hace en los Security Groups.

Aceptado: el UDP del juego debe estar abierto a internet; lo protege la contraseña de Valheim.

2 · Identidad y acceso

Lambdas con el rol LabRole; instancia con LabInstanceProfile (sin claves en el código).
Ningún usuario IAM ni access key creados por el proyecto.

API: clave compartida en el header x-api-key, validada en la Lambda; sin clave → 401.
Panel: HTML público de solo lectura; los datos solo se ven con la clave.
Servidor de juego: contraseña obligatoria (parámetro NoEcho, nunca en texto plano).
Webhook de Discord tratado como secreto (parámetro, no en el código).
Lifecycle hook y Run Command: solo los invoca la Lambda, con el rol del lab.

Limitación del Learner Lab: no se pueden crear roles con permisos mínimos; se usa LabRole.

3 · Datos en reposo

EBS del servidor cifrado (AES-256, clave de AWS) y descartado al apagar.
S3 mundos: SSE-S3, versionado, Block Public Access, lifecycle de 30 días, DeletionPolicy Retain.
S3 panel: solo lectura pública de un HTML sin datos sensibles.
DynamoDB: cifrado en reposo por defecto, Point-in-Time Recovery (35 días).
Backups del mundo cada 20 min y al apagar (lifecycle hook); restauración al arrancar.
Logs del servidor y de las Lambdas en CloudWatch Logs, retención 14 días.

4 · Datos en tránsito

HTTPS/TLS en todo el plano de control: panel, API, SDK de las Lambdas y de la instancia.
S3 y DynamoDB desde la instancia por Gateway Endpoints: no salen a internet.
Session Manager y Run Command por el agente SSM (HTTPS saliente, sin puertos abiertos).
Juego: protocolo UDP de Valheim sobre Steam Networking, con contraseña del servidor.
Webhook de Discord por HTTPS.

Guardas de costo y límites

ASG máx 1 por servidor · MaxRunningServers en la API (429) · apagado por inactividad 30 min · apagado de seguridad 04:00 · tags owner:team y cost-center · billing alarm (consola) · sin NAT ni Interface Endpoints

Figura 5. Seguridad por capas: lo implementado y los riesgos aceptados.

6. RESILIENCIA

El sistema tolera la pérdida de la instancia o de una AZ sin intervención, con una pérdida máxima de 20 minutos de juego (el intervalo de backup) y un corte de 2 a 4 minutos. No tolera la pérdida de la región.

- **Caída de una AZ.** El ASG abarca dos AZ y el NLB tiene un nodo en cada una: lanza otra instancia en la otra AZ, el UserData restaura el mundo y los jugadores vuelven a la misma dirección. S3, DynamoDB, Lambda, API Gateway y EventBridge son regionales.
- **Caída de la región.** Sin servicio. La mitigación sería replicar el bucket de mundos y desplegar los templates en otra región; no se hizo por costo y alcance.
- **Base de datos.** DynamoDB replica en tres AZ y tiene Point-in-Time Recovery. El dato crítico es el mundo, en S3 con versionado y backup cada 20 minutos; el stack de datos tiene protección contra borrado y el bucket `DeletionPolicy: Retain`.
- **Usuarios y logs.** Los jugadores se reconectan a la misma dirección y pierden como mucho 20 minutos (nada si el apagado fue limpio). La consola de Valheim y las Lambdas escriben en CloudWatch Logs, que sobrevive a la instancia; lo que falla va a dos DLQ en SQS.
- **Otras fallas.** Si falla una Lambda, EventBridge reintenta y el mensaje queda en la DLQ; si falla la creación de un stack, CloudFormation deshace los recursos y el panel muestra el motivo; si se rompe la tabla o se borra un mundo, se vuelve con PITR o con la versión anterior en S3.

7. ESCALABILIDAD

La unidad de escala es el servidor completo. El sistema escala en el tiempo (de 0 a 1 por servidor), en cantidad (un stack por servidor) y en tamaño (tipo de instancia por parámetro). El plano de control escala solo.

DIMENSIÓN	CUÁNDO	CÓMO	LÍMITE
Temporal (0 a 1)	Se prende desde el panel; 30 min sin jugadores; cron de las 04:00	Capacidad deseada del ASG en 1 o 0	`MaxSize: 1`
Horizontal	Un mundo llega a 10 jugadores o se quiere otro mundo	Nuevo servidor en el panel: otro stack	10 creados y 8 prendidos; la API responde 429
Vertical	CPU alta o falta de memoria	Parámetro `InstanceType`; aplica en el próximo arranque	Hasta `large` en el lab

DIMENSIÓN	CUÁNDO	CÓMO	LÍMITE
Plano de control	Siempre	Lambda, DynamoDB on-demand, API Gateway y NLB elásticos	Throttling de 10 req/s

- **Capacidad y límites.** 8 servidores prendidos × 10 jugadores (límite del juego) = 80 jugadores. Los techos los pone el entorno: 9 instancias concurrentes en el Learner Lab y 50 NLB por región. Los límites están en el código para que un error no deje varios t3.large prendidos; fuera del lab se piden cuotas sin cambiar la arquitectura.
- **ECS/EC2 y TCO.** Con Fargate un servidor equivalente cuesta cerca del doble por hora y no resuelve el estado; ECS sobre EC2 agrega una capa para un contenedor por instancia. Operar con 3 horas de juego por día cuesta unos 25 USD por mes (13 con Elastic IP), frente a 65 de un t3.large prendido 24 horas.

8. COSTOS

El costo depende casi solo de las horas que el servidor está prendido; el plano de control cuesta centavos. Precios on-demand de us-east-1.

COMPONENTE	PRECIO	CUÁNDO SE PAGA
EC2 t3.large + EBS gp3 20 GB + IP pública	0,09 USD/h	Solo con el servidor prendido
Network Load Balancer	0,0225 USD/h (16,4 USD/mes)	Mientras exista el stack del servidor
S3, snapshot de la AMI, CloudWatch	~0,3 a 1,3 USD/mes	Siempre
Lambda, DynamoDB, API Gateway, EventBridge, SQS, SSM	~0 USD	Dentro del free tier

- **Escenarios.** Todo desplegado con el servidor apagado, unos 0,6 USD por día (casi todo el NLB); un mes con 3 horas de juego por día, unos 25 USD con NLB o 13 con Elastic IP; el mismo t3.large prendido 24/7, unos 65 USD; un hosting comercial de Valheim, de referencia, 10 a 15 USD por mes.
- **Control.** Se evitaron NAT Gateway (33 USD/mes), Interface Endpoints (7 USD/mes cada uno) y RDS (15 USD/mes). Tags `owner:team` y `cost-center` en todos los recursos, billing alarm, apagado de seguridad diario y los límites de la sección 7.

9. DESPLIEGUE

Cuatro templates que se crean en orden, porque cada uno exporta valores que importa el siguiente. Los únicos pasos manuales son subir el `index.html` al bucket del panel y el `04-servidor.yaml` a `templates/` en el bucket de mundos. El despliegue completo tarda menos de 15 minutos.

#	STACK	QUÉ CREA	TIEMPO
1	`valheim-01-red`	VPC, 2 subredes públicas y 2 privadas, IGW, rutas, Gateway Endpoints	1 min
2	`valheim-02-datos`	Bucket de mundos (versionado, lifecycle, eventos), bucket del panel, DynamoDB con PITR; termination protection	1 min
3	`valheim-03-control`	6 Lambdas con sus logs, HTTP API, 6 reglas de EventBridge, 2 DLQ, dashboard	2 min
4	`valheim-srv-<nombre>`	SG, NLB UDP, Launch Template, ASG 0/1 con lifecycle hook, alarma, log group. Lo crea el panel o la consola	3 min

- **Parámetros y lab.** Clave de la API, webhook, límites y cron en el stack de control; nombre, mundo, contraseña (`NoEcho`), tipo de instancia y AMI opcional en el de servidor. Con la AMI base el UserData instala Docker y descarga el juego (4 a 8 minutos); con la AMI propia arranca en uno. En el Learner Lab no se crean roles (`LabRole`), las instancias llegan hasta `large` y las sesiones de 4 horas detienen las instancias: se cierra con todo apagado.

10. PRUEBAS

Se probó por capas y de punta a punta en el Learner Lab, con el panel, la consola de AWS y el Discord del grupo como evidencia.

PRUEBA	RESULTADO	ESTADO
Infraestructura como código	Cuatro stacks de cero, dos veces, `CREATE_COMPLETE` al primer intento. Drift detectado; termination protection rechaza el borrado.	**Pasó**
Seguridad de la API	Clave incorrecta → 401.	**Pasó**
Prender y arranque	ASG a 1, instancia lanzada, En línea con la dirección del NLB en 4 minutos.	**Pasó**
Conexión al juego	Unirse por IP con la dirección y la contraseña.	**Pasó**

PRUEBA	RESULTADO	ESTADO
Contador de jugadores	La métrica `PlayersOnline` refleja los jugadores conectados. La primera versión publicaba 0 con jugadores adentro; se corrigió (hallazgo más abajo).	**Corregido**
Backup periódico	Zip en `backups/<servidor>/` cada 20 minutos y evento en el historial.	**Pasó**
Apagado por inactividad y apagado limpio	30 minutos sin jugadores → `idle-stop` → ASG a 0; lifecycle hook, Run Command y mundo actualizado en S3.	**Pasó**
Avisos por Discord	Un mensaje por cada paso.	**Pasó**
Crear servidor desde el panel	202, tarjeta Creando, stack creado; Apagado y listo en 2 minutos.	**Pasó**
Validaciones y límites	Contraseña en el nombre → 400; nombre repetido → 409; `MaxServers` alcanzado → 429.	**Pasó**
Eliminar servidor	Confirmación, tarjeta Eliminando, `DELETE_COMPLETE` en 2 minutos; el mundo queda en S3. Con el servidor prendido la API responde 409.	**Pasó**

- **Hallazgo corregido.** Con `SERVER_PUBLIC=false` Valheim no respondía las consultas de estado y la métrica publicaba 0 con jugadores conectados; la alarma apagó el servidor a los 30 minutos. Se dejó el servidor público (con contraseña) y la métrica no publica nada si no puede leer el estado, de modo que un fallo del contador no apaga una partida.



Figura 6. Servidor creado desde el panel, en Apagado dos minutos después del pedido, con los eventos creating y stopped en el historial.

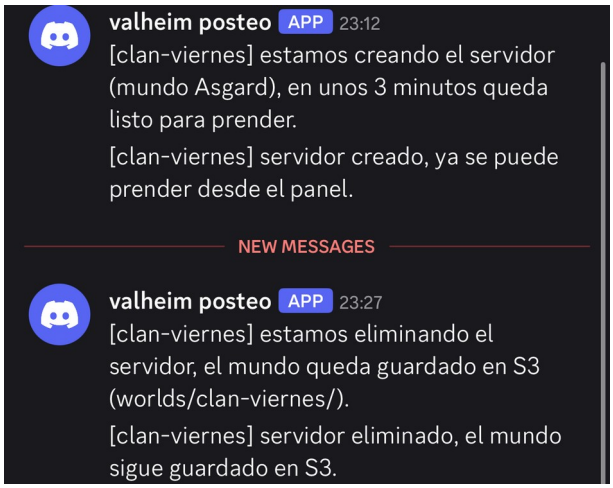


Figura 7. Avisos en el Discord del grupo al crear y eliminar un servidor desde el panel.

11. LIMITACIONES

LIMITACIÓN	MOTIVO
No tolera la caída de la región (todo vive en us-east-1)	Alcance y costo del proyecto
Clave compartida para el panel, sin usuarios	Simplicidad para el curso
Hasta 20 min de juego perdidos si la instancia muere de golpe	Intervalo de backup de 20 minutos
Roles IAM amplios (`LabRole`, `LabInstanceProfile`)	El Learner Lab no permite crear roles
NLB caro para una sola instancia (16 USD por mes)	DNS fijo, health checks y registro automático

12. CONCLUSIONES

Separar lo que administra (serverless) de lo que corre el juego ordenó el diseño: lo que administra cuesta centavos y escala solo, y lo único que cuesta dinero está prendido mientras alguien juega. El ASG como interruptor, con el mundo en S3, hizo descartable a la instancia y dio autorreparación; CloudFormation es además la API con la que el panel crea y elimina servidores.